

The USSD Channel

How Africa's Talking drives `ATUsersUssdApI`, why a Micromart Fintech customer is offered Micromart Africa's menu, and what it takes to turn one lender's USSD app into something every Micro Eazy client can dial.

Subject `skegode/ATUsersUssdApI` · .NET 9 Evidence live read, Micromart ServiceSuite Status in production

§ 0

What was found

There is no hardcoded 3002. The string does not appear in the C# and it does not appear in any of the eight stored procedures the service calls. Micromart's report is accurate about the symptom and wrong about the cause, and that distinction matters, because the actual causes are four separate defects that a search-and-replace would not have touched.

What a Micromart Fintech customer sees when they choose *Borrow Loan* is a six-item category menu — BUSINESS LOAN-DAILY, SCHOOL FEES LOAN, SALARY LOAN and the rest. That list is Micromart Africa's product taxonomy. It is shown to the Fintech customer because the query behind it has no entity filter and the table behind *that* has no entity column. Four of those six categories contain nothing the Fintech book can sell. The customer picks one, gets a header with no options beneath it, and concludes the service is broken.

That is the presenting complaint. Underneath it the service has a deeper problem: it has no concept of which book it is serving. The entity is re-derived from the phone number at four separate call sites, by two different rules, one of which returns a literal zero for exactly the customers who most need it to be right. The PIN — the only credential in the system — is read and written with no entity filter at all.

VERIFIED LIVE

Every figure in this document was read from Micromart's ServiceSuite database on 2 September 2026 through the Connected Suite's read-only connection. The queries are reproduced in the appendix.

15,472

borrowers hold a USSD PIN across the three books this service can reach. It is live and in daily use.

4 of 6

loan categories offered to a Fintech customer contain no product that customer can be sold.

44

phone numbers exist in more than one book. Every one of them can neither borrow nor safely set a PIN.

0

loans carrying the USSD service's own signature have ever been booked on the Fintech book. 25 carry it on 3002.

The defects, in the order they should be fixed

ID	DEFECT	EFFECT ON A FINTECH CUSTOMER	SEVERITY
F1	The category menu is read from a global table with no entity filter	Sees 3002's taxonomy; four of six choices are dead ends	CRITICAL
F2	The PIN is read and written with no entity predicate	Setting a PIN overwrites the PIN on every book that shares the number	CRITICAL
F3	Multi-book customers are asked for products under entity 0	Empty product menu, always	CRITICAL
F4	The level router has no multi-book branch at level 5	Multi-book customers can never complete a loan application	CRITICAL
F5	Lender selection is not scoped to the operator's own entities	Two Micromart numbers are offered Axe Capital's book by name	CRITICAL
F6	The EntityUnits gate inside the product proc is invisible to the app	529 Fintech borrowers in the wrong branch see no products at all	HIGH
F7	The KES 3,000 minimum is enforced in SQL and discarded in C#	A KES 2,000 application is confirmed and silently dropped	HIGH
F8	The endpoint is unauthenticated and the PIN has no attempt limit	A four-digit PIN is brute-forceable by anyone who can reach the URL	HIGH
F9	The STK response is discarded; the log writer is not concurrency-safe	"Wait for the M-Pesa prompt" when no prompt was sent; intermittent 500s	MEDIUM

Sections 1 and 2 explain the channel and the code, because the fixes in section 4 only make sense once the shape of a USSD session is clear. Section 3 is the evidence. Sections 5 and 6 answer the strategic question — reuse this repository, or start a clean one — and the answer is *both, in that order*.

How Africa's Talking USSD actually works

USSD is not a protocol you implement. It is a protocol Safaricom implements, Africa's Talking translates, and you answer. Your entire surface area is **one HTTP endpoint that receives a form post and returns a string**. Everything difficult about USSD follows from how little that string is allowed to be.

1.1 · The channel

Africa's Talking leases a service code from the operator and sublets a slice of it to you. Micromart's customers dial something of the form `*384*NNNN#`: `*384#` is the shortcode Africa's Talking operates on Safaricom, and `NNNN` is the channel assigned to a single application. You register one callback URL against that channel in the AT dashboard. From then on, every keypress by every customer on that code arrives at that URL.

That last sentence is the whole tenancy story, and section 6 turns on it: **the channel is one-to-one with the callback URL, and the callback URL is one-to-one with a deployment**. Two lenders on one deployment means two channels pointing at the same URL — which is fine, and is exactly how this becomes reusable, provided the code reads the field that tells them apart.

1.2 · The four fields

The gateway posts `application/x-www-form-urlencoded` with four fields. There are no others, and there is no body, no JSON, and no header worth reading.

FIELD	WHAT IT CARRIES
sessionId	Opaque string, stable for the life of one dialogue, new on every dial. The only handle you get on "this conversation".
serviceCode	The code the customer dialled. The tenant discriminator — see §6.1. This service reads it once, to write it to a log file.
phoneNumber	MSISDN in <code>+2547XXXXXXX</code> form. Asserted by the network, not by the handset — the one genuinely trustworthy field.
text	Everything the customer has typed this session, joined by <code>*</code> . Empty on the first request.

1.3 · CON and END

You reply with `text/plain`. The first four characters decide what the handset does with the rest:

THE DIAL STRING

`*384*NNNN#` is an Africa's Talking sublet. A lender who wants their own `*XYZ#` must apply to the operator directly — months, and a fee. For every client after Micromart, an AT channel is the practical choice.

```
CON Enter your PIN
└ input box shown · session
  stays open
```

The customer can type. Your next request will carry what they typed.

```
END Your loan balance is 4,500
└ message shown · session
  destroyed
```

No input box. The dialogue is over and the sessionId will never return.

There is no third option. There is no way to send a message and keep the session open without asking for input, no way to push a screen the customer did not ask for, and no way to reopen a session you ended. `END` is irreversible, which is why every error path in this service — including recoverable ones like a mistyped PIN — costs the customer the whole dialogue.

1.4 · The cumulative string is the state

This is the idea everything else rests on. Africa's Talking keeps no state on your behalf beyond the session's existence. Instead, **every request re-sends the complete input history**, asterisk-delimited, from the top.

A customer who dials in, enters PIN `1234`, chooses *Borrow Loan*, picks category 2, product 1, and types 5000 generates *six* HTTP requests, each carrying strictly more than the last:

request	text=	what you must return
1	(empty)	CON Welcome ... enter your PIN
2	1234	CON 1. Check Balance / 2. Pay / 3. Borrow
3	1234*3	CON Select Category
4	1234*3*2	CON Select Product Code
5	1234*3*2*1	CON Enter Amount
6	1234*3*2*1*5000	END Your request has been received

Two properties follow directly. First, **navigation depth is `text.Split('*').Length`** — the service can route purely on how many selections have been made, which is precisely what `HandleMainFlow` does. Second, **the PIN is re-sent in plaintext on every request** and is re-verified against a bcrypt hash each time. Six requests, six bcrypt verifications, each deliberately costly. At work factor 12 that is real latency inside a session with a hard timeout.

CONSEQUENCE

The service is stateless by construction. It never has to remember anything — but it also has to re-validate the PIN, re-resolve the customer, and re-establish which book it is in, on every single keypress. This service does exactly that, and it is where the entity bugs live.

1.5 · The screen budget

A USSD screen is capped at **182 characters** by GSM 03.38, and handsets in the field routinely render less. Overflow is not wrapped or paginated — it is truncated, silently, by equipment you do not control.

Micromart's category screen, measured against the live table, is **159 characters**. It fits, with 23 characters of headroom. Adding a seventh product category — a perfectly ordinary business request — overflows it and truncates the menu mid-item on some handsets. Any reusable version of this service has to treat 182 as a budget it enforces, with pagination as a first-class feature rather than something bolted on when a client complains.

	chars
CON Select Category:	21
1) BUSINESS LOAN-DAILY	22
2) BUSINESS LOAN-WEEKLY	23
3) BUSINESS LOAN-MONTHLY	24
4) SCHOOL FEES LOAN	19
5) SALARY LOAN	15
6) ASSET LOAN	14
0. Back	8
00. Exit	10
total	159 / 182

The live category screen. Every one of these six lines is drawn from a table with no entity column — see F1.

1.6 · Sessions end without telling you

The operator enforces a maximum session duration — on Safaricom Kenya this is conventionally around three minutes, and it should be confirmed for Micromart's specific channel rather than assumed. When it expires, **no callback is made**. Your service does not learn that the customer walked away; the requests simply stop.

The same is true of the per-request timeout. Africa's Talking waits a few seconds for your HTTP response and then abandons the session. This is the single most important operational constraint on the design, and it rules out exactly the thing a loan application wants to do: call a credit bureau, run a score, wait. Anything slow must be queued and answered with `END ... is being processed` — which is, correctly, what this service does.

1.7 · What the gateway does not give you

YOU DO NOT GET	WHAT THAT MEANS FOR THE DESIGN
Authentication	AT posts unsigned. Anyone who learns the callback URL can post any <code>phoneNumber</code> they like. Mitigation is IP allow-listing plus a shared secret in the path or a header — see F8.
Idempotency	A retried or duplicated request is indistinguishable from a real one. A booking endpoint reached at the last level of a menu must be idempotent on <code>sessionId</code> , or a network hiccup books two loans.
Delivery confirmation	You never learn whether your <code>END</code> screen was displayed.
Rich input	Digits and <code>*</code> . No letters in practice, no confirmation dialogs, no back button — which is why <code>0</code> and <code>00</code> have to be invented in application code.

How this service implements it

`ATUsersUssdApi` is a single-controller .NET 9 web API — 936 lines across 16 files, one endpoint, no tests, no authentication, and a database contract of four tables and eight stored procedures. It is small enough to hold in your head, which is a genuine virtue and the main argument for porting it rather than rewriting from a specification.

2.1 · The request lifecycle

Every keypress from every customer enters at `POST /ussd/service` and runs the same six steps:

- 01 **Normalise the MSISDN.** `PhoneHelper.Normalize` strips non-digits, takes the last nine, and prefixes `254`. Correct, total, and — importantly — the same transformation the stored procedures apply internally, so the two agree.
- 02 **Log, if there is input.** A file append to `C:\Logs\ussd\`. See F9: the writer is a singleton with no lock.
- 03 **Check PIN status.** One query. If the borrower has no PIN yet, the request is diverted into the enrolment flow and never reaches the menu.
- 04 **First screen, or main flow.** Empty `text` means the customer has just dialled: greet them by lender name and ask for a PIN.
- 05 **Split, normalise back/exit, validate the PIN.** The PIN is `parts[0]`, re-verified with `bcrypt` on every request.
- 06 **Route on depth.** `parts.Length` selects one of five level handlers. This is the whole navigation model.

2.2 · The one piece of server state, and why it barely matters

There is a `UssdSession` table, written by `sp_ussdsession`, which increments a level counter per `(sessionId, phoneNumber)`. It is used for one purpose only: **PIN enrolment**, where the customer has typed a new PIN and a repeat, and the service needs to know which of the two prompts it is on.

Every other screen ignores it entirely and derives position from `text`. This is not an inconsistency worth fixing so much as one worth noticing: the enrolment flow could be derived from `text` too, and then the table — and the write on every enrolment keypress — disappears.

2.3 · Back and exit

USSD has no back button, so `0` and `00` are conventions the application invents. `NormalizeParts` implements them by *rewriting history*: it walks the `parts` array and pops the previous selection whenever it meets a `0`.

AN HONEST STRENGTH

Steps 1, 4 and 6 are well judged. Routing on `parts.Length` rather than on a persisted session cursor is the right call for a stateless gateway, and it is why the service survives an AT retry without corrupting anything.

```
// text = "1234*3*2*0" → parts = [1234, 3] → back at the categories
for (int i = 1; i < rawParts.Length; i++) {
    if (rawParts[i] == "0") {
        if (list.Count > 1) list.RemoveAt(list.Count - 1); // back
        else return (list.ToArray(), true); // nothing to go back to → exit
    }
    else list.Add(rawParts[i]);
}
```

This is the cleverest code in the repository and it is correct. It has one consequence worth naming: **0 is reserved at every level, including the amount prompts**. A customer who types 0 as a repayment amount goes back a screen instead. That is defensible — zero is not a valid amount — but a customer typing 00 as an amount exits the session, and neither behaviour is explained on screen.

2.4 · Two customer shapes, and only one of them works

sp_checkCustomerCompomany counts the borrower rows carrying this phone number and returns a sentinel:

RETURNS	MEANING	MENU PATH
-1	Not registered	END — visit a branch
-2	One book	Straight to Balance / Pay / Borrow. Works.
-3	More than one book	"Select Lender" first, then the same three. Broken at three separate points — F3, F4, F5.

Note what the count is *not* scoped by. It counts rows across every entity on a shared ServiceSuite instance — Micromart Africa, Micromart Fintech, Axe Capital, and anyone else who lands there. That is the root of F5.

2.5 · The database contract

The service owns almost no logic. Nearly everything of consequence lives in stored procedures, which means **a fix that touches entity scoping is a SQL change as much as a C# change** — and a change to procedures that other systems may also call.

PROCEDURE OR TABLE	ENTITY-SCOPED?	WHAT IT DOES, AND WHAT IS WRONG WITH IT
Borrowers	HAS ENTITYID	Identity, PIN, branch (EntityUnit), limit. The app queries it with TOP 1 and no ORDER BY .
Products	HAS ENTITYID	Also carries EntityUnits , a comma-joined branch list — a second scope the app knows nothing about.
ProductCategories	NO ENTITYID	The table at the heart of F1. Six global rows. Nothing about it can be entity-scoped without going through Products .
UssdSession	n/a	Enrolment cursor only.
sp_checkCustomerCompomany	NO	Counts borrower rows across <i>all</i> entities → F5.
sp_CompanyName	TOP 1	Names the lender on the welcome screen via an unordered TOP 1 . Can greet a dual-book customer with the wrong brand.
GetCustomerCompanies	NO	Lists every entity the number appears under, joined to BsEntity . Returns the raw EntityId as the menu key.
sp_ussdGetproducts	YES, PLUS UNITS	Filters on EntityId , CategoryId , IsActive <i>and</i> a CHARINDEX over EntityUnits → F6.
sp_GetolbOrLoanLimit	YES	Correctly scoped on both PhoneNumber and EntityId . The model the rest should follow.
sp_UssdInsertLoan	YES	Enforces a KES 3,000 floor and returns a decline code the app discards → F7.
sp_GetUssdStkParams	YES	Per-entity Daraja credentials. Rows exist for both 3002 and 3005 — the payment path is correctly tenanted.

The pattern worth extracting

sp_GetolbOrLoanLimit and sp_GetUssdStkParams are both correct: they take @EntityId and they use it. The procedures that are wrong are wrong in the same way — they were written when there was one book. Every fix in §4 is a variation on giving the other procedures the parameter these two already have.

The entity problem, measured

Micromart runs two books on one ServiceSuite instance: **3002, Micromart Africa Limited**, the established SME lender, and **3005, Micromart Fintech**, where the Micro Eazy products live. Both are reachable from the same USSD deployment and the same database connection. The entity is not a label on the data — it is an identity boundary, and the same phone number in the two books can belong to two different people.

BOOK	BORROWERS	WITH A USSD PIN	LOANS	PRODUCTS	ON THE USSD MENU
3002 · Micromart Africa Limited	142,757	12,473	276,518	34	27
3005 · Micromart Fintech	17,022	2,968	61,549	3	2
3003 · Axe Capital (same instance)	2,454	31	—	37	0

BOTH BOOKS ARE LIVE The most recent loan on 3002 was booked at 11:20 and on 3005 at 10:05, on the morning this was written. Whatever is changed here is changed underneath a working credit channel. Axe Capital shares the instance but not the shortcode — it appears here because §3 F5 shows that today it is reachable from it.

F1 The category menu is not scoped to anything

CRITICAL

`GetLoanCategories` is four lines and contains the complaint in full:

```
using var cmd = new SqlCommand("SELECT ID, CategoryName FROM ProductCategories",
    con);
```

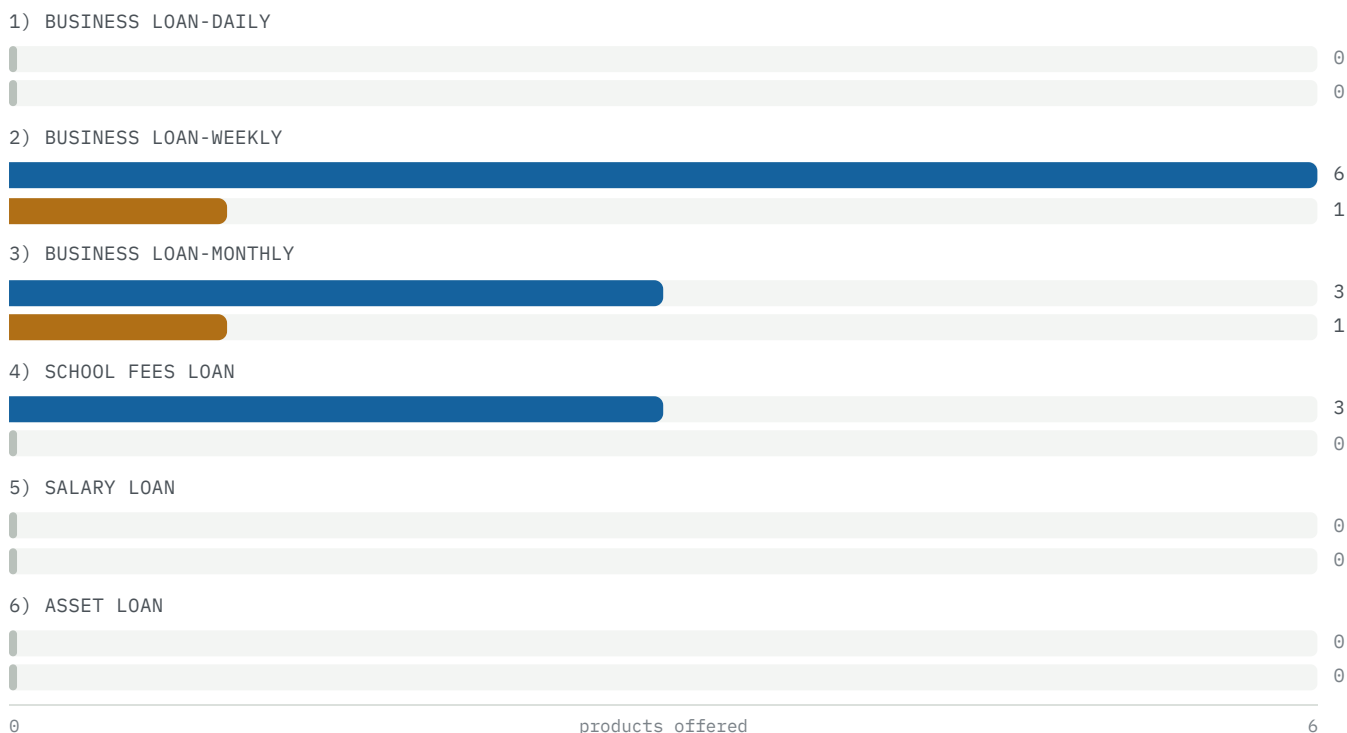
No WHERE, no `@EntityId`, no join to `Products`. And it could not have one: `ProductCategories` has six columns — `ID`, `CategoryName`, and four audit fields — and **no entity column at all**. It is a global taxonomy that predates the second book.

So every customer of every lender on the instance is shown the same six categories. Replaying `sp_ussdGetproducts` against the live tables for a real, PIN-enrolled borrower in each book shows what is actually behind each of those six doors:

Products a customer is actually offered, by category

Replay of `sp_ussdGetproducts` for a PIN-enrolled borrower in each book. A row with no bar is a category shown on the menu with nothing behind it.

■ 3002 · Micromart Africa ■ 3005 · Micromart Fintech



A Fintech customer is offered six categories and can buy from two of them. They are shown `SCHOOL FEES LOAN` because Micromart Africa sells school fees loans. That is the literal, precise sense in which "a customer in 3005 gets the products of 3002" — not the product list, which is correctly filtered, but the taxonomy above it, which is not filtered at all.

The main book is not well served either: three of its six categories are also empty. Nobody should ever be shown a menu item that leads nowhere, and today both books are.

F2 The PIN is read and written across every book at once

CRITICAL

Enrolment writes the new PIN like this:

```
new SqlCommand("UPDATE Borrowers SET USSDPIN = @pin WHERE PhoneNumber = @phone", con);
```

No `EntityId`. No `TOP 1`. For a phone number present in two books, **this overwrites the credential on both** — including the row belonging to a different person. Validation has the mirror-image defect, reading `SELECT TOP 1 UssdPin ... WHERE PhoneNumber = @mobile` with no entity predicate and no `ORDER BY`, so it authenticates against whichever book the query plan happens to reach first.

Forty-four numbers are in this position today. That is a small number and a serious one: the fix is one `AND EntityId = @EntityId`, and the consequence of leaving it is that one customer's PIN change silently changes another customer's PIN.

THE OVERLAP, EXACTLY

44 distinct phone numbers span more than one book: 43 across Micromart 3002 and 3005, 2 across Micromart 3002 and Axe Capital 3003. One number appears in all three, which is why the pairs sum to 45.

F3 Multi-book customers are asked for the products of entity zero

CRITICAL

`GetEntityId.EntityId` has two behaviours. For a single-book customer it looks the entity up from `Borrowers`. For a multi-book customer it returns `selectedEntity` — the value the caller passed in — unchanged. Which makes the caller's argument load-bearing:

```
// ServiceController.GetProducts — reached by BOTH customer shapes
int entityId = _getEntityId.EntityId(req.phoneNumber,
                                   customerStatus,
                                   0,           // ← the entity
                                   constr);

// For a MultiCompany customer, EntityId() returns this argument
// unchanged. So the proc runs as sp_ussdGetproducts @EntityId = 0
// and returns nothing — while the lender the customer actually
// selected sits unread in parts[1].
```

`CheckBalance` passes the same literal zero. It happens to be harmless there — the multi-book balance path is handled separately, one branch earlier — but it is the same latent mistake in a second place.

This is the closest thing in the repository to the complaint as reported. It is not a hardcoded 3002; it is a hardcoded 0. The effect is not that the customer gets the wrong book's products, but that they get a header and no products at all.

F4 A multi-book customer can never finish a loan application

CRITICAL

The two customer shapes need different menu depths, because the multi-book customer spends one selection choosing a lender:

DEPTH	SINGLE BOOK	MULTI BOOK
1	Main menu	Select lender
2	Category list	Main menu
3	Product list	Category list
4	Enter amount	Product list
5	Book the loan	Enter amount — not implemented
6	—	Book the loan — no handler exists

`HandleLevel5` opens with `if (customerStatus == CustomerStatus.SingleCompany)` and has no corresponding multi-book branch, so it falls through to `END Invalid selection`. There is no `HandleLevel6` at all; the router's default arm returns `END Invalid input`.

Combined with F3, the multi-book borrow flow is broken twice over — it shows an empty product list, and had the list been populated, the application could not have been submitted. The 43 customers who hold accounts in both Micromart books cannot borrow over USSD at all.

F5 One lender's customers are offered another lender's book

CRITICAL

`sp_checkCustomerCompomany` counts borrower rows for a phone number with no entity restriction, and `GetCustomerCompanies` then lists every entity that number appears under, joined to `BsEntity` for the display name. Neither knows which lender's USSD code was dialled — because, as §1.2 noted, `serviceCode` is never read.

Two Micromart numbers also exist in Axe Capital's book. Those customers, dialling Micromart's code, are shown a lender menu that names Axe Capital, and selecting it queries `sp_GetolbOrLoanLimit` against entity 3003 — returning an Axe Capital loan balance to whoever is holding that SIM.

```
CON Select Lender
3002. MICROMART AFRICA LIMITED
3005. MICROMART FINTECH
0. Exit
```

What the code produces today. Note that the menu keys *are* the raw entity ids — the customer is expected to key `3005`, not `2` — because `HandleLevel2` parses the keypress directly as an `EntityId`.

```
CON Select Lender
3002. MICROMART AFRICA LIMITED
3003. AXE CAPITAL
3005. MICROMART FINTECH
0. Exit
```

What two Micromart customers are shown. A different lender's brand, reachable from Micromart's shortcode, with that lender's balance behind it.

On a shared `ServiceSuite` instance this is a tenancy breach rather than a cosmetic bug, and it will get worse with every lender added to the instance, not better.

F6 The branch gate inside the product proc strands 529 Fintech borrowers

HIGH

`sp_ussdGetproducts` applies a scope the C# has no idea exists. It looks up the borrower's branch and requires the product to list that branch:

```
SET @EntityUnit = (SELECT EntityUnit FROM Borrowers
                   WHERE EntityId = @EntityId AND PhoneNumber = @Phone)

SELECT ID, ProductName, UssdOrder FROM Products
WHERE IsActive = 1 AND CategoryId = @CategoryId AND EntityId = @EntityId
      AND CHARINDEX(',' + CONVERT(VARCHAR(MAX), @EntityUnit) + ', ',
                        CONCAT(',', EntityUnits, ',')) > 0
```

Micromart Fintech's two USSD-visible products both carry `EntityUnits = '0,129'`. Its borrowers are distributed across four branches:

BRANCH (ENTITYUNIT)	BORROWERS	LISTED ON THE PRODUCTS?
129	16,493	YES — PRODUCTS APPEAR
35	417	NO — EMPTY MENU
25	108	NO — EMPTY MENU
1	4	NO — EMPTY MENU

529 Fintech borrowers — 3.1% of the book — reach the product menu and are shown nothing, in the two categories that do have products. Nothing in the application layer can detect this: the proc returns zero rows and the app renders a header. Worse, if the borrower has no row for the entity being

queried at all, `@EntityUnit` is `NULL`, the `CHARINDEX` comparison evaluates to `NULL`, and the predicate silently excludes everything — which is the mechanism by which F3's entity zero produces an empty menu rather than an error.

This is a data fix, not a code fix, and it should be Micromart's call: either add branches 1, 25 and 35 to the two products' `EntityUnits`, or move those 529 borrowers into branch 129. But the application should also stop rendering an empty menu — see §4.3.

F7 The KES 3,000 floor is enforced, then thrown away

HIGH

`sp_UssdInsertLoan` declines anything under KES 3,000 and returns a row saying so. Its own header comment is explicit that the app ignores it — and it does:

```
-- in the proc
IF (@Principal < 3000)
BEGIN
    SELECT 0 AS LoanID, 9 AS Code,
           'Amount applied must be at least 3000' AS Response;
    RETURN
END

// in the controller
cmd.ExecuteNonQuery();           // ← the decline row is discarded
return "END Your request has been received and is being processed.";
```

A customer applying for KES 2,000 is told their application succeeded. Nothing is booked, nothing is queued, and no one is notified. The floor is not surfaced anywhere on the amount screen either, so there is no way for the customer to have known.

The same procedure accepts a `@SelectedPeriod` parameter that the USSD flow never collects, so every USSD loan takes whatever tenor `sp_InsertLoan` defaults to. That is worth a conversation with Micromart before it is treated as a defect — it may be deliberate.

F8 The endpoint is open and the PIN has no attempt limit

HIGH

`POST /ussd/service` has no authentication filter, no IP restriction, and no shared secret. Africa's Talking does not sign its posts, so something has to stand in for that, and nothing does. Anyone who discovers the URL can post any `phoneNumber` with any `text`.

They still need the PIN — but the PIN is four digits, there is no lockout, no attempt counter, and no rate limit. Ten thousand posts exhausts the keyspace for a chosen customer, after which the attacker can read balances, trigger STK prompts, and submit loan applications in that customer's name. Bcrypt at work factor 12 is the only thing slowing this down, and it was chosen to protect the hash at rest, not to serve as a rate limiter.

Three mitigations, in order of value: allow-list Africa's Talking's egress addresses at the firewall or reverse proxy; move the callback to an

unguessable path segment carrying a shared secret; and add a per-MSISDN attempt counter that locks the PIN after five failures. The first two are configuration and can be done this week.

F9 Two smaller defects that will show up under load

MEDIUM

The STK response is discarded. `InitiateStkPush` returns "Failed to retrieve STK parameters." when the entity has no Daraja row, and returns Safaricom's raw response body otherwise. `InitiatePayment` ignores the return value entirely and reports success unless an exception was thrown. A rejected STK push — wrong shortcode, expired passkey, throttled — reads to the customer as "Wait for the M-Pesa prompt". Both Micromart books currently have valid rows, so this is latent rather than active, and it will become active the first time a new lender is onboarded without one.

The log writer is not concurrency-safe. `WriteLog` is registered as a singleton and opens the day's file with `FileShare.Read` on every request that carries input — which is most of them. Two concurrent sessions means the second request throws `IOException` and returns a 500, which Africa's Talking renders to the customer as a dead session. This is invisible at low volume and guaranteed at high volume. It also writes to `C:\Logs\ussd\`, a hardcoded path that ties the service to a Windows host.

Relatedly, `Payments` constructs `new HttpClient()` per STK push, exhausting sockets under load. All three are standard fixes: a lock or a logging framework, and an injected `IHttpClientFactory`.

What the ledger says

Every loan this service books passes through `sp_UssdInsertLoan`, which stamps it `CreatedBy = 483993` — a synthetic user id standing in for the channel. That signature makes USSD-originated lending countable, and the count is the shortest possible statement of the problem:

A BY-PRODUCT WORTH COLLECTING

All 25 carry `Loans.ChannelUsed = 1` — the first hard evidence for any value of a column the analytics studio renders as raw codes, for want of a lookup table. It proves containment, not equivalence: channel 1 holds 99,415 loans on 3002 alone.

LOANS STAMPED	CREATEDBY = 483993	3002 · MICROMART AFRICA	3005 · MICROMART FINTECH
Booked through USSD		25	0
Most recent		2 Sep 2026, 11:42	—

READ CONSERVATIVELY `sp_UssdInsertLoan` was revised recently to track the BuySimu build, so this counts loans booked since that revision rather than every USSD loan ever written. The asymmetry is the point, not the absolute number.

The channel works. It has never worked for the Fintech book.

Twenty-five applications reached the ledger, the most recent at 11:42 on the morning this was written — so the endpoint, the PIN gate, the product lookup and the booking path all function. Not one of them landed on 3005. That is exactly what F1, F3 and F6 predict, and it is what Micromart reported from the other end of the phone.

The fix

Every defect from F1 to F6 is the same defect wearing different clothes: **the entity is re-derived, late, at each call site, instead of being resolved once and carried**. This is precisely the shape of the bug already fixed in the borrower PWA, where four different notions of "entity" coexisted in one app and only one of them was right. The remedy there was a single module that owns the answer. The remedy here is the same.

4.1 · Resolve once, at the top, and carry it

The controller already has everything it needs by line 4 of `HandleUssd`: the normalised MSISDN, the service code, and the text. Resolve the book there, once, and pass it down. Nothing below should ever call `GetEntityId` again.

```
/// The book this session is in. Resolved once per request, never re-derived.
public sealed record UssdContext(
    string Phone,
    int EntityId, // the book. Never 0, never guessed.
    int EntityUnit, // the branch - sp_ussdGetproducts needs it
    string LenderName,
    bool MultiBook);

/// Resolution order, most authoritative first:
/// 1. the entity the customer explicitly selected this session
/// 2. the entities this SERVICE CODE may serve ($6.1 - closes F5)
/// 3. the borrower's own row, when there is exactly one
/// If none of those yields exactly one book, ASK. Never fall through to a
/// literal, and never to an unordered TOP 1.
```

Two rules make the difference between this holding and quietly rotting again:

1. **An unresolved entity is an error, not a zero.** The reason F3 was invisible for so long is that entity 0 produces an empty menu, and an empty menu is indistinguishable from "this customer has no eligible products". Fail loudly instead.
2. **No TOP 1 without an ORDER BY.** There are three of them today, in `GetEntityId`, `Login.ValidatePin` and `sp_CompanyName`. Each returns whichever row the query plan reaches first, which is stable until the day an index changes and then is not.

WHY THIS MATTERS MORE THAN THE INDIVIDUAL FIXES

Patch F1–F6 one by one and they come back with the next feature. Introduce `UssdContext` and there is no longer a place to put the bug: there is one entity in scope and it was resolved before the routing began.

4.2 · The C# changes

ID	WHERE	CHANGE
F2	ServiceController.ConfirmNewPin	Add <code>AND EntityId = @entityId</code> to the <code>UPDATE</code> . One line, and the highest-value line in this table.
F2	Login.ValidatePin / UssdPinStatus	Take the entity as a parameter; add it to both <code>WHERE</code> clauses; drop the bare <code>TOP 1</code> .
F3	ServiceController.GetProducts	Read the selected entity from <code>parts[1]</code> for multi-book customers instead of passing <code>0</code> . Better: take it from <code>UssdContext</code> and delete the argument.
F4	HandleLevel5 / HandleLevel6	Add the multi-book branch at level 5 (prompt for amount) and a level 6 handler (book the loan). Both are near-copies of the existing single-book arms with the index shifted by one.
F7	ServiceController.HandleLevel5	<code>ExecuteScalar</code> instead of <code>ExecuteNonQuery</code> ; branch on the returned code; surface the decline text. Also state the minimum on the amount screen.
F9	InitiatePayment	Inspect the string <code>InitiateStkPush</code> returns and the Safaricom <code>ResponseCode</code> before promising a prompt.
F9	WriteLog	Replace with <code>ILogger</code> and a rolling file sink, or take a lock. Remove the hardcoded <code>C:\Logs\ussd\</code> .
F9	Payments	Inject <code>IHttpClientFactory</code> ; stop constructing <code>HttpClient</code> per call.
F8	Program.cs / hosting	IP allow-list, secret path segment, PIN attempt counter. Also reconsider <code>UseHttpsRedirection()</code> — if AT is registered against an <code>http</code> URL, the redirect can cost the POST body.

4.3 · The SQL changes

F1 and F5 cannot be fixed in C# alone. Both need the procedures to learn about the entity.

```
-- F1: categories that actually contain something this customer can be sold.
-- New proc rather than an edit, so nothing else that reads ProductCategories
-- is affected. ProductCategories keeps no EntityId; Products supplies the scope.
CREATE PROCEDURE dbo.sp_UssdGetCategories
    @EntityId INT, @Phone VARCHAR(20)
AS
BEGIN
    DECLARE @Unit INT = (SELECT EntityUnit FROM Borrowers
                        WHERE EntityId = @EntityId
                        AND PhoneNumber = @Phone);

    SELECT DISTINCT c.ID, c.CategoryName
    FROM   ProductCategories c
    JOIN   Products p ON p.CategoryId = c.ID
    WHERE  p.EntityId = @EntityId
    AND    p.IsActive = 1
    AND    p.UssdOrder IS NOT NULL
    AND    CHARINDEX(',' + CONVERT(VARCHAR(MAX), @Unit) + ',',
                    CONCAT(',', p.EntityUnits, ',')) > 0
    ORDER BY c.ID;
END
```

Note what this does beyond fixing F1. Because it applies the same `EntityUnits` and `IsActive` predicates as `sp_ussdGetproducts`, a **category can no longer be listed unless the product list behind it will be non-empty** — which closes the user-visible half of F6 for free. A Fintech customer in branch 35 sees no categories at all rather than six that lead nowhere, and the service can say so plainly instead of showing a header over emptiness.

On the live data this reduces the Fintech category menu from six items to two — `BUSINESS LOAN-WEEKLY` and `BUSINESS LOAN-MONTHLY` — and the main book's

from six to three. It also drops the screen from 159 characters to well under half the budget.

```
-- F5: only the books this operator is allowed to serve.
-- @AllowedEntities is a comma list supplied by the app from the service-code
-- registry ($6.1), so Axe Capital can never appear on Micromart's shortcode.
ALTER PROCEDURE dbo.GetCustomerCompanies
    @phone VARCHAR(20), @AllowedEntities VARCHAR(200)
AS
BEGIN
    SET @phone = '254' + RIGHT(@phone, 9);
    SELECT bs.ID, bs.EntityName
    FROM    Borrowers b
    JOIN    BsEntity bs ON bs.ID = b.EntityId
    WHERE   b.PhoneNumber = @phone
           AND CHARINDEX(',' + CONVERT(VARCHAR(20), b.EntityId) + ',',
                               ',' + @AllowedEntities + ',') > 0
    ORDER BY bs.EntityName;
END
```

`sp_checkCustomerCompomany` needs the same treatment — its count must be restricted to the allowed entities, or a customer with one Micromart account and one Axe account will still be misclassified as multi-book and sent down the broken path.

Finally, the lender menu should present *ordinals*, not entity ids. A customer should key `1` or `2`, and the service should map that back to the entity. Keying `3005` works today only because `HandleLevel2` parses the keypress as an `EntityId` directly, which is also what makes the menu unsafe to reorder.

4.4 · What not to change

Leave these alone until someone confirms them

The EntityUnits data. Adding branches 1, 25 and 35 to the Fintech products makes 529 borrowers eligible for credit they are not eligible for today. That is a lending decision, not a bug fix. Surface it to Micromart and let them decide.

The @SelectedPeriod default. USSD loans currently take `sp_InsertLoan`'s default tenor. Collecting a period on the menu changes the terms of every USSD loan booked afterwards.

The Triple-DES / MD5 Decipher. It is weak, and it is what the existing `Stkparams` ciphertext was written with. Replacing the algorithm requires re-encrypting live Daraja credentials for every entity; do that as its own migration, never as part of this work.

This is the same restraint applied to `registerFCM` in the PWA, where `PhoneNumber: fcmToken` looked like swapped fields but could not be corrected without knowing what the backend had actually been storing. Fixing what is provably wrong and flagging what merely looks wrong is the difference between a safe change and an outage.

Reuse this repository, or start a new one?

Both — in that order. Patch this one in place for Micromart, because it is carrying live traffic and 15,472 enrolled PINs. Build a new one for everybody else, because this one cannot be made multi-tenant without becoming a different program.

5.1 · Why not simply fix this one and reuse it

OBSTACLE	WHY IT BLOCKS REUSE
It is somebody else's repository	<code>skegode/ATusersUssdApi</code> is upstream, the same relationship the ecosystem has with <code>PBXAuthentication</code> : tracked read-only, never pushed to. Every ecosystem change would be a pull request into a repo whose release cadence we do not control.
It is a single-tenant deployment artifact	One connection string in <code>appsettings.json</code> , one log path, one publish profile pointing at a named person's Desktop. Adding a lender means a second deployment, a second server, a second set of credentials to rotate.
Tenancy is not a parameter anywhere	<code>serviceCode</code> arrives on every request and is written to a log file. There is no registry, no per-tenant configuration, and no place to put one that is not a new module.
The menu is hard-coded C#	Five level handlers with the menu tree expressed as <code>switch</code> arms and string concatenation. A lender who wants a different menu order needs a code change and a redeploy.
No tests, and a live channel underneath	276,518 loans on one book and 61,549 on the other, both booking today. Restructuring it is not a refactor, it is an outage risk.

5.2 · The recommendation

- 01 Patch `ATusersUssdApi` in place, minimally.** F2, F3, F5 and F1 — the four that are actively wrong for Micromart — plus the F8 configuration mitigations. No restructuring, no `UssdContext` refactor if it cannot be done safely, no new dependencies. Two to three days including a rehearsal against a copy of the database. This is the deliverable Micromart is asking for.
- 02 Stand up `micro-eazy-ussd` as a new ecosystem repository.** Multi-tenant from the first commit, registered in `ecosystem/registry.json` like every other system, with the tenant model of §6 rather than one bolted on later. Port the parts of this service that are genuinely good — and there are several — and rebuild the rest.
- 03 Prove it on the second lender, not on Micromart.** Point a new Africa's Talking channel at it for whichever client comes next. Micromart migrates only once the new service has run somebody else's book for a month. Their channel is the one thing in this picture that must not break.

5.3 · What to port, what to leave

PORT AS-IS	PORT WITH CHANGES	LEAVE BEHIND
PhoneHelper.Normalize — correct and total	The five-level menu tree → menu-as-data (§6.3)	WriteLog and C:\Logs\ussd\
NormalizeParts back/exit semantics	The PIN gate → add entity scope and a lockout	The UssdSession table — derive enrolment position from text
Routing on parts.Length	The STK call → check the response	GetEntityId in its entirety — replaced by the resolver
The CON/END discipline	sp_GetolbOrLoanLimit and sp_GetUssdStkParams, which are already correctly scoped	LoanApplication.InsertLoan, which is dead code calling a proc nothing invokes
Queue-and-confirm for anything slow		The MD5 + Triple-DES Decipher, in the new service only, with fresh credentials

5.4 · What to build it in

TypeScript, as its own small service. The reason to rebuild at all is to inherit the ecosystem's tenant discipline, and that discipline is already written down in `connected-suite/src/lib/enterprise/connections.ts` — an org registry mapping a slug to an `EntityId` and a connection string, resolved server-side, with the client never seeing a raw entity id. Rewriting that in C# to keep the .NET stack means maintaining two copies of the one thing that must never disagree.

Against that, one real constraint: **USSD has a latency budget of a few seconds and no room for an extra hop.** The Connected Suite reaches ServiceSuite over a relay when direct TDS is unavailable, and a relay round trip inside a USSD session is a session that dies. The new service should hold a direct TDS connection from a host with a route to the database, with the relay as failover only — the same "list of roads" the suite already implements, ordered differently because the deadline is harder.

If that route cannot be guaranteed, the argument flips: keep .NET, deploy it beside ServiceSuite on IIS, and port `connections.ts`'s *model* rather than its code. The tenancy design in §6 is language-independent; it is the discipline that has to carry over, not the syntax.

Making it reusable across the ecosystem

A Micro Eazy client buying "the USSD product" should get a shortcode, a row in a registry, and their own menu — with no fork, no redeploy, and no second server. Three changes get there, and the first one is already sitting in the request payload.

6.1 · `serviceCode` is the tenant key, and it is free

Africa's Talking already tells you, on every single request, which shortcode the customer dialled. One channel per lender, one callback URL for all of them, and the tenant is `request.serviceCode`. This service captures the field in `UssdRequest` and uses it exactly once — to write it to a log line.

```
// The registry. One entry per lender — mirrors ORGS in connections.ts.
export const CHANNELS = {
  "*384*4001#": { slug: "micromart",
    entities: [3002, 3005],
    conn: "SERVICESUITE_CONN_MICROMART" },

  "*384*4002#": { slug: "axe",
    entities: [3003, 3004],
    conn: "SERVICESUITE_CONN_AXE" },

  "*384*4003#": { slug: "njb",
    entities: [3],
    conn: "SERVICESUITE_CONN_BUYSIMU" },
};

// Resolution becomes a lookup instead of a guess, and F5 becomes impossible:
// a book that is not in this channel's `entities` cannot be reached from it,
// however many books the customer's phone number appears in.
```

Two things fall out of this immediately. The lender-selection menu can be intersected with the channel's entities, which closes F5 by construction rather than by a `WHERE` clause someone has to remember. And a single deployment serves every client, because the connection string, the entity list and the branding all hang off the same lookup.

6.2 · One entity model, shared with the rest of the ecosystem

The registry above is deliberately the same shape as `ORGS` in `connections.ts`: a slug, a default `EntityId`, an env var holding the connection string, an env var that overrides the entity per deployment. The USSD service should not invent a second one. Where the two must differ is that a channel maps to a *list* of entities rather than one, because a lender with two books genuinely has two and the customer picks.

The rule the whole ecosystem already follows, and which this service should adopt verbatim: **one connection per organisation, scoped by `EntityId`**. Both Micromart books live on one SQL Server; the entity is what scopes the read, not the credential.

ONBOARDING, AFTER THIS CHANGE

Request a channel from Africa's Talking, point it at the existing callback, add one registry row, add the connection string to the environment. No deployment, no code.

6.3 · Menus as data

The five level handlers encode Micromart's menu as C# control flow. The next lender will want a different order, different wording, Swahili, or a product tree one level shallower. Express the tree as data instead — a list of screens, each with a prompt, a source (static, or a query), and a target — and the level handlers collapse into one interpreter that walks the tree using `parts.Length` as the cursor. The screen budget of \$1.5 becomes a property the interpreter enforces, with pagination for free.

This is also what makes the `0 / 00` convention testable. Today it is correct; there is nothing stopping the next edit from breaking it.

6.4 · Next steps, in order

	STEP	EFFORT	UNBLOCKS
01	Confirm the AT channel, session timeout and callback URL for Micromart's shortcode	hours	Everything — and it is currently undocumented
02	Patch F2 (PIN entity scope) and F3 (entity zero)	1 day	The credential bug and the empty product menu
03	Add <code>sp_UssdGetCategories</code> ; point the app at it	1 day	F1 and the visible half of F6 — the reported complaint
04	Scope <code>GetCustomerCompanies</code> and <code>sp_checkCustomerCompomany</code> to allowed entities	1 day	F5, the cross-lender leak
05	IP allow-list plus a secret path segment on the callback	hours	F8's first two mitigations — configuration only
06	Take the <code>EntityUnits</code> question to Micromart for a decision	—	The 529 stranded Fintech borrowers
07	Fix F4 (multi-book levels 5 and 6) and F7 (the KES 3,000 floor)	2 days	The 43 dual-book customers; silent application drops
08	Create <code>micro-eazy-ussd</code> ; register it in <code>ecosystem/registry.json</code>	1 day	The rebuild has somewhere to live
09	Build the channel registry and the entity resolver; port the good parts	1 week	Multi-tenancy, and F1–F6 by construction
10	Menu-as-data interpreter with a 182-character budget and pagination	1 week	Per-lender menus with no redeploy
11	Run the next client's channel on the new service; leave Micromart where it is	—	Proof, without risking a live book

The one thing to decide first

Steps 02–07 patch a repository we do not own, on a live channel, for one client. Steps 08–11 build the thing every client after Micromart will use. They are independent and can run in parallel, but only if it is agreed up front that **the patches are deliberately throwaway** — no refactoring, no architecture, nothing that needs porting later. Otherwise the patch work quietly becomes the rebuild, on somebody else's repository, under a live shortcode.

Appendix — the verification queries

Every figure in §3 came from these, run against Micromart's ServiceSuite through the Connected Suite's read-only connection on 2 September 2026. They are reproduced so the findings can be re-checked, and so the same checks can be run against any other book before that lender is onboarded.

WHICH BOOKS THIS DEPLOYMENT CAN REACH, AND HOW MANY ARE ENROLLED

```
SELECT EntityId,
       COUNT(*) AS borrowers,
       SUM(CASE WHEN UssdPin IS NOT NULL AND UssdPin <> ''
                THEN 1 ELSE 0 END) AS withPin
FROM   Borrowers
GROUP BY EntityId ORDER BY COUNT(*) DESC;
```

F2 / F5 – NUMBERS THAT EXIST IN MORE THAN ONE BOOK

```
SELECT a.EntityId AS entA, b.EntityId AS entB, COUNT(*) AS phones
FROM   Borrowers a
JOIN   Borrowers b ON b.PhoneNumber = a.PhoneNumber AND a.EntityId < b.EntityId
WHERE  a.PhoneNumber IS NOT NULL AND LEN(a.PhoneNumber) > 8
GROUP BY a.EntityId, b.EntityId ORDER BY COUNT(*) DESC;
```

F1 / F6 – WHAT EACH CATEGORY ACTUALLY OFFERS, PER BOOK

```
-- Replays sp_ussdGetproducts' own predicates for a real enrolled borrower.
-- Swap 3005 for any entity to check a book before onboarding it.
SELECT c.ID AS categoryId, c.CategoryName,
       (SELECT COUNT(*) FROM Products p
        WHERE p.IsActive = 1 AND p.CategoryId = c.ID AND p.EntityId = 3005
          AND CHARINDEX(',' + CONVERT(VARCHAR(MAX),
            (SELECT TOP 1 EntityUnit FROM Borrowers
             WHERE EntityId = 3005 AND UssdPin IS NOT NULL)) + ',',
            CONCAT(',', p.EntityUnits, ',')) > 0) AS productsOffered
FROM   ProductCategories c ORDER BY c.ID;
```

F6 – THE BRANCH DISTRIBUTION AGAINST THE PRODUCTS' ENTITYUNITS

```
SELECT EntityUnit, COUNT(*) AS borrowers FROM Borrowers
WHERE EntityId = 3005 GROUP BY EntityUnit ORDER BY COUNT(*) DESC;

SELECT ID, ProductName, CategoryId, UssdOrder, IsActive, EntityUnits
FROM   Products WHERE EntityId = 3005;
```

READING THE PROCEDURES RATHER THAN ASSUMING THEM

```
SELECT OBJECT_DEFINITION(OBJECT_ID('dbo.sp_ussdGetproducts'));
-- and the other seven: sp_checkCustomerCompomany, sp_CompanyName,
-- GetCustomerCompanies, sp_GetolbOrLoanLimit, sp_UssdInsertLoan,
-- sp_GetUssdStkParams, sp_ussdsession
```

Source repository: [skegode/ATUsersUssdApi](#) at [343d10b](#) · 16 files, 936 lines · .NET 9 · Dapper, RestSharp, BCrypt.Net, Microsoft.Data.SqlClient. Database evidence read through [connected-suite/src/lib/enterprise/mssql.ts](#) under the read-only guard. No writes were made to any book.